

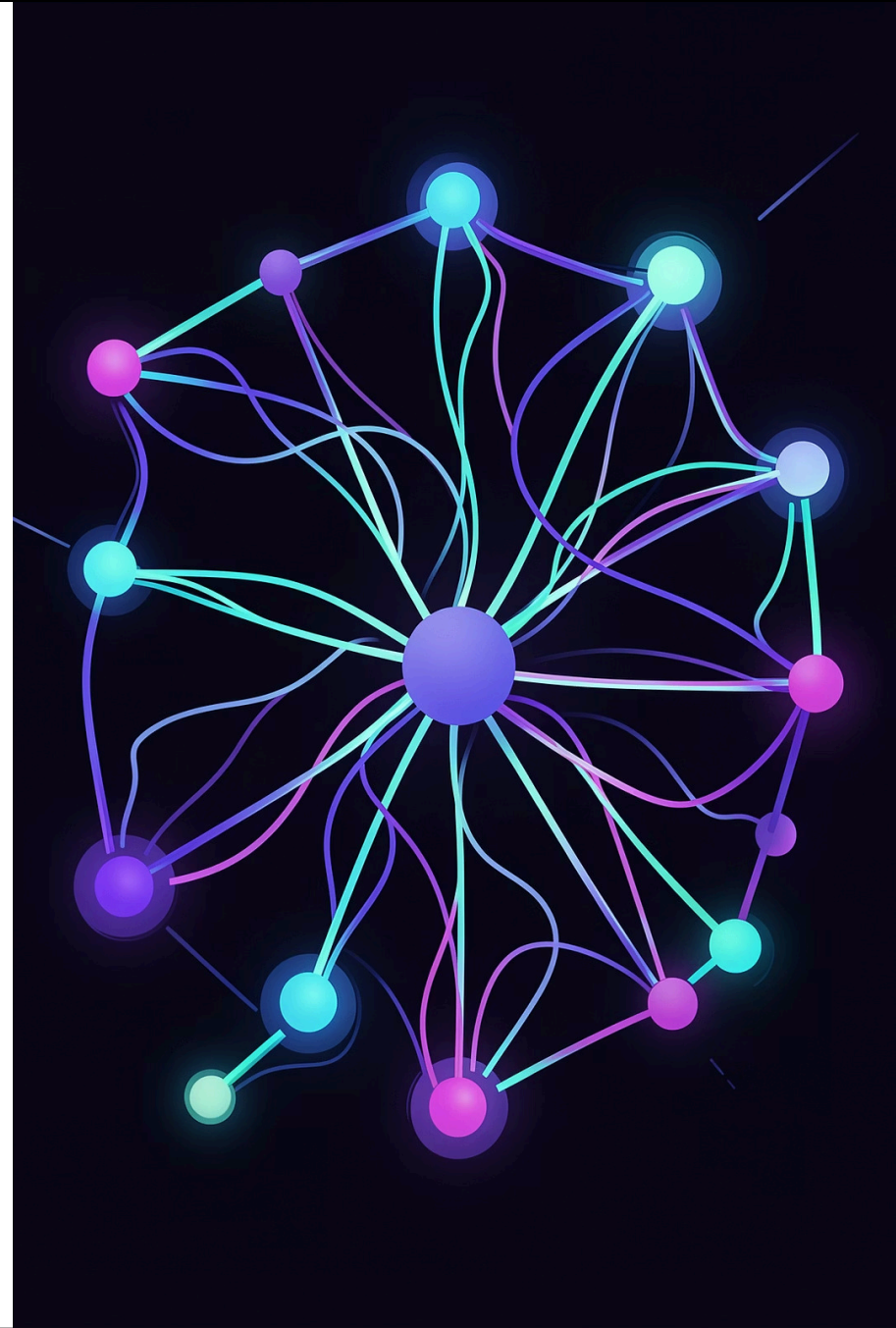
AI-Driven DevOps: How LLMs and Agents Are Changing Software Delivery

KCD Texas 2026 · Austin · May 15, 2026

Kedar Kulkarni — Senior DevOps architect @ Apple · ex-VMware, Red Hat · co-founder of Ansible Tower Config-as-Code (1M+ downloads)

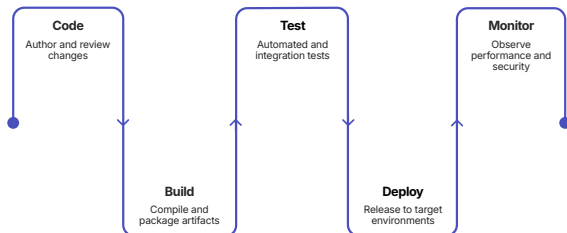
Today, in 25 minutes: what's hurting → what's actually new since last year → two demos → what to do Monday morning.

📄 *Disclaimer: Views are my own, not Apple's.*



Software Delivery Reality

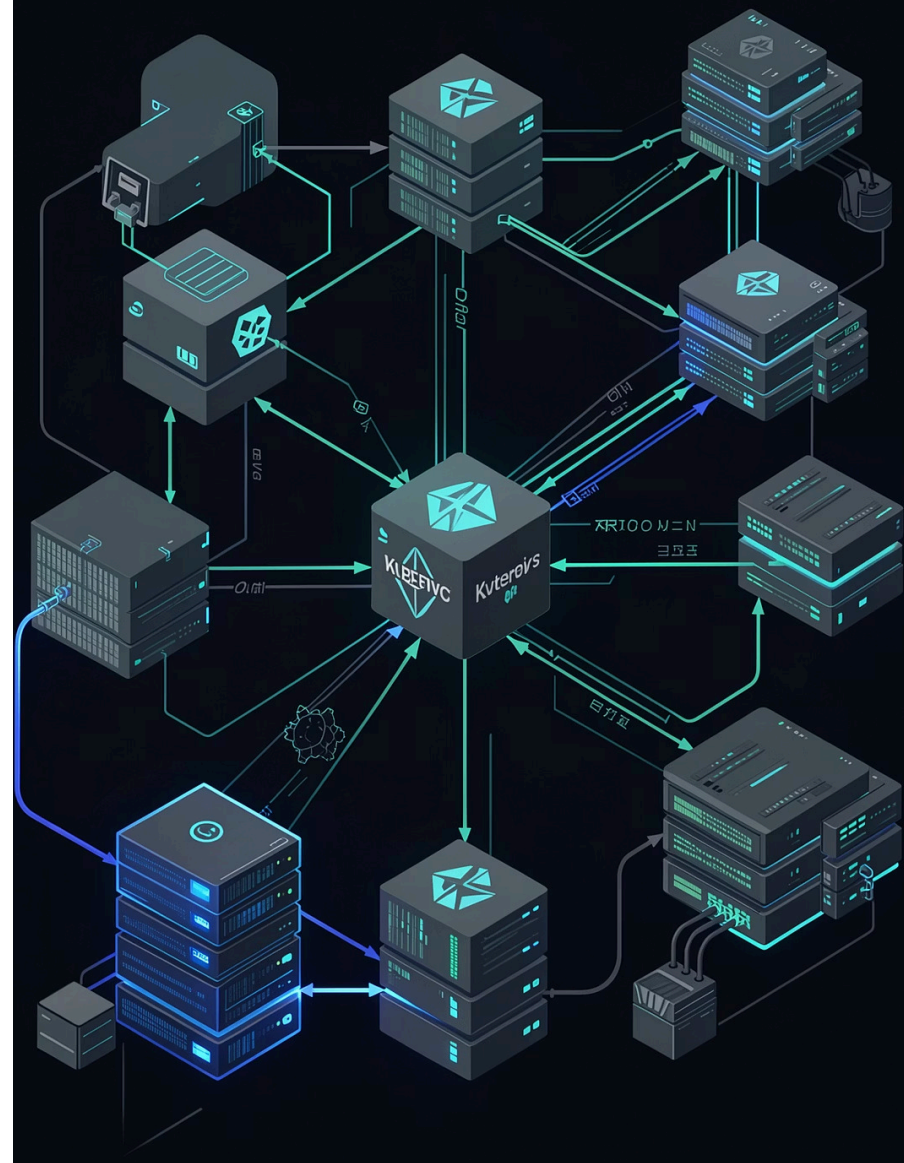
Traditional View



Actual Reality in 2026

- Multi-cloud + hybrid + edge environments
- Microservices sprawl and service-mesh complexity
- GPU-scheduled AI workloads sharing clusters with traditional services
- Security and compliance as continuous concerns
- Incident response across 15+ tools

 Kubernetes has won — but "won" means it's now the operating layer for everything, including the AI workloads that used to run somewhere else.



Toil = Cognitive Load (and Its Real Cost)

Toil isn't busy work — it's three categories of cognitive load:

Pattern Recognition

Scanning logs, correlating metrics, "this looks like last Tuesday's incident"

Context Switching

Finding the runbook, recalling rare procedures, translating between tool dialects

Knowledge Access

Who owns this service? Where's the Slack thread? What does the previous on-call know that I don't?

The cost varies by: experience · domain · context · access



Same incident: **5 minutes** for someone who's seen it before, **2 hours** for someone who hasn't.

The bill comes due as: pager fatigue · 15+ panes of glass · P0s outside your expertise at 2am · brittle automation no one wants to touch

Decoding AI for DevOps Teams

LLMs

- Advanced pattern matching for text and code
- Like a senior engineer who's read everything, but has no hands
- Good at: explaining, summarizing, generating, translating
- DevOps terms: smart search + content generation

AI Agents

- LLM + tools + a decision loop (Observe → Reason → Act)
- Can actually *do* things, not just describe them
- Good at: multi-step tasks with feedback, investigation, routine remediation
- DevOps terms: an engineer who never sleeps but needs supervision

 **The bridge between these two worlds is MCP.**

MCP — The Protocol That Changed Everything

Model Context Protocol (MCP)

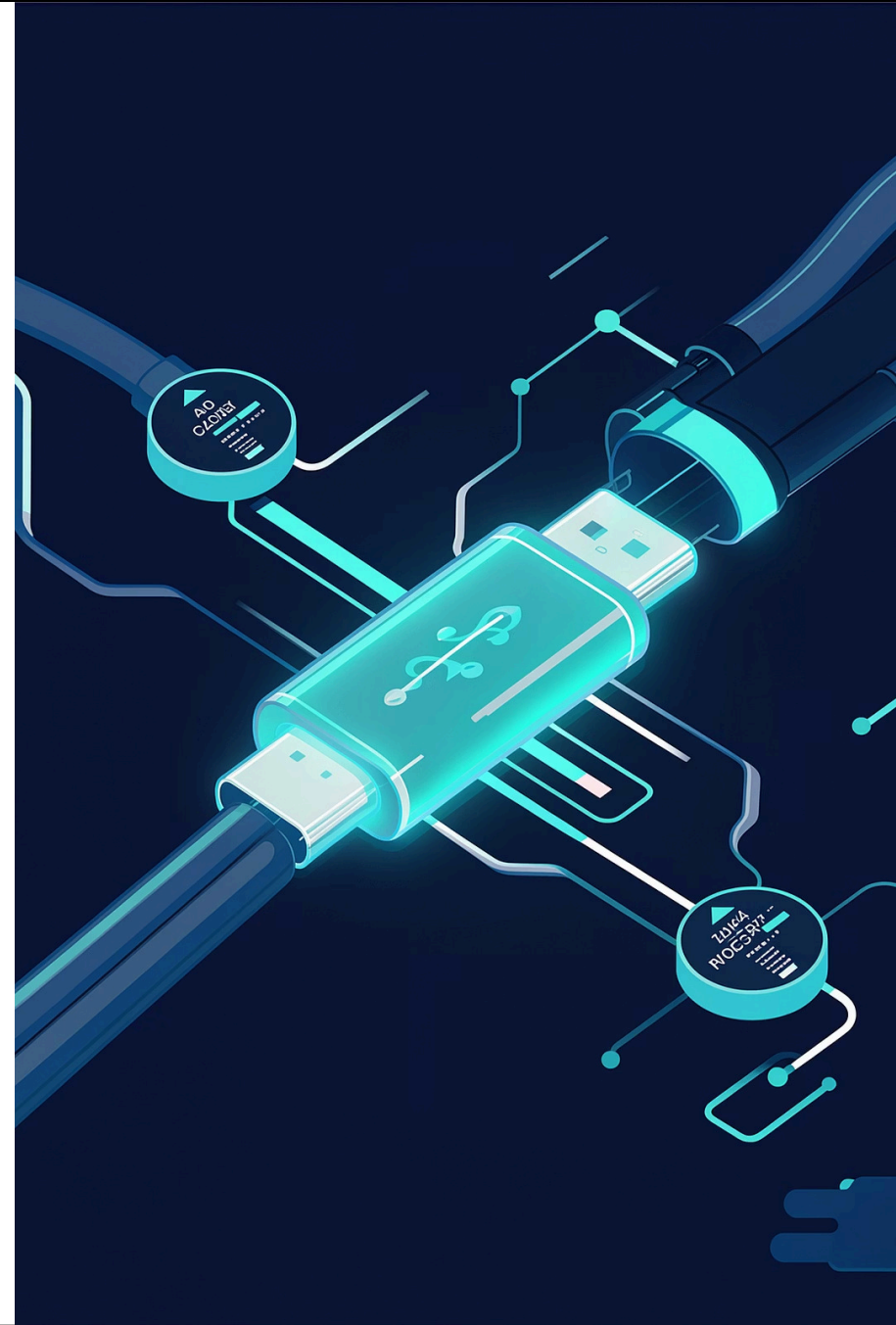
- Open standard introduced by Anthropic in late 2024
- Adopted by OpenAI (Apr 2025), Microsoft (Jul 2025), AWS (Nov 2025)
- Donated to the Linux Foundation (Agentic AI Foundation) in Dec 2025
- 10,000+ public MCP servers, ~97M monthly SDK downloads (Anthropic, Mar 2026)

Think of it as: **USB-C for AI agents.**
One connector, many tools.

Why It Matters to DevOps

Before MCP, every AI tool integration was a bespoke wrapper. Now, one MCP server for Kubernetes works with Claude, ChatGPT, Cursor, Windsurf, or whatever you're using next quarter.

✓ **DevOps implication:**
Credentials stay local, RBAC-scoped, auditable. You can give an agent *read-only Kubernetes* without handing over your API keys.



The CNCF-Native Agent Stack

Two binaries, your kubeconfig, an LLM key — that's the whole setup. **This is where most teams should start.**

Today's Working Loop

- ❑ Two binaries, your kubeconfig, an LLM key — that's the whole setup. **This is where most teams should start.**

📘 **Honest framing:** If you're scripting `k8sgpt analyze | kubectl-`
`ai ...` today, Kagent is what that loop wants to grow up into.

Tomorrow's Production Setup: Kagent



CNCF Sandbox Framework

Runs *inside* the cluster as
Kubernetes-native workloads



Declarative CRDs

Agents defined as GitOps-
friendly CRDs. Ships with MCP
servers for: Kubernetes, Istio,
Helm, Argo, Prometheus,
Grafana, Cilium



OpenTelemetry Tracing

Every reasoning step is traced
and observable



k8sgpt (CNCF Sandbox)

Scans cluster, explains errors in plain English



kubectl-ai

Natural language → kubectl commands with confirmation

AI as the Great Equalizer



Junior Engineers

Instant institutional knowledge, plain-language log explanations, guided troubleshooting.



Senior Engineers

Offloads pattern-matching, compresses investigation time, acts as a thinking partner.



Non-Native English Speakers

Translates technical jargon both directions.

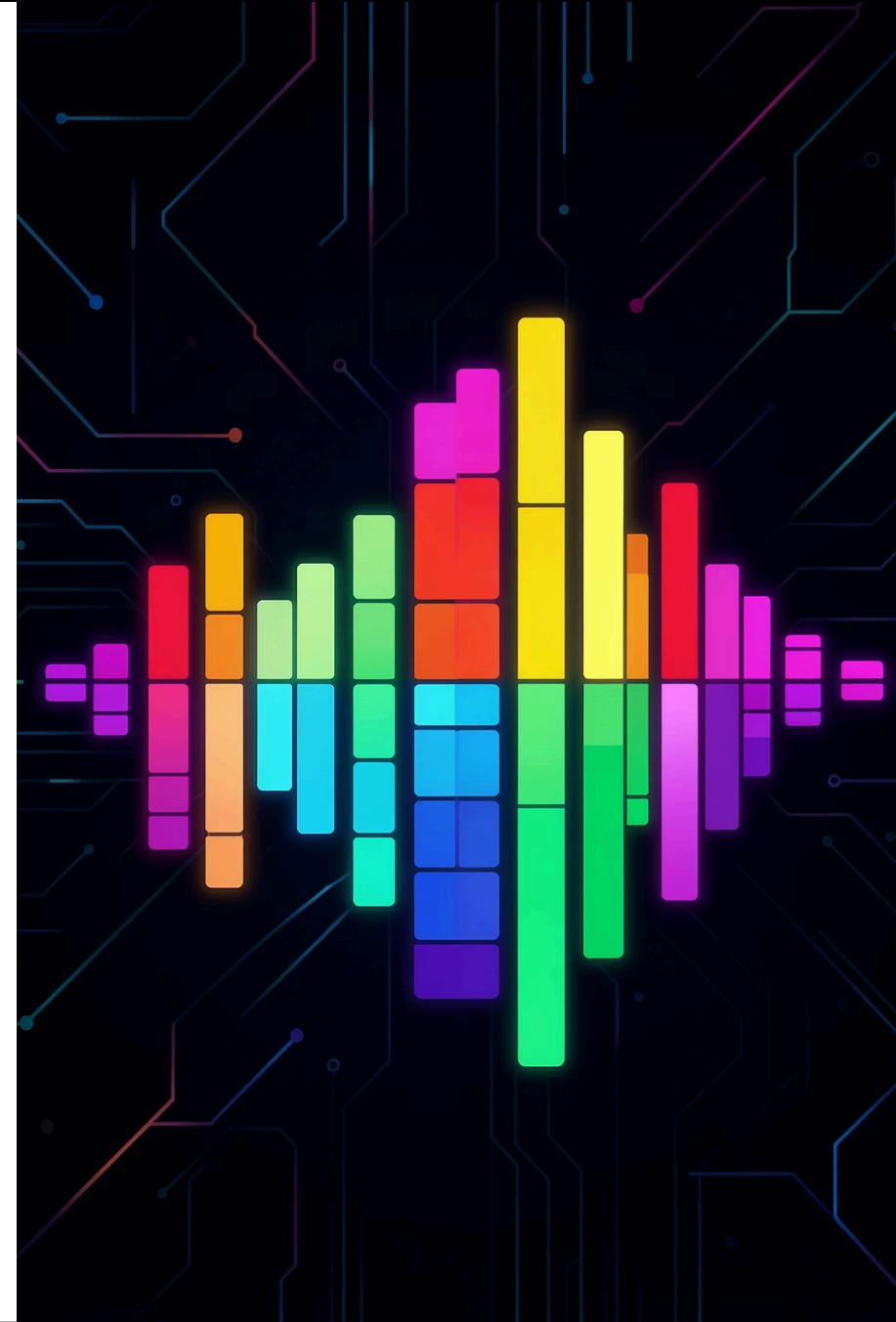


Different Learning Styles

Visual learners get diagrams, sequential learners get step-by-steps.



The equalizer cuts both ways — it also flattens the gap between teams with great internal docs and teams without. That changes hiring, onboarding, and what "seniority" even means.



Intelligent CI/CD — The Safety Spectrum

1

Advisory

Safe today — do this now

- RCA summaries on CI failures
- terraform plan risk review before apply
- PR risk scoring and release notes
- Flaky test classification

2

Semi-Autonomous

Pilot with guardrails

- Deployment verification (pre/post telemetry comparison)
- Auto-generated fixes for linter/security findings
- Flaky test auto-retry with reasoning

3

Autonomous

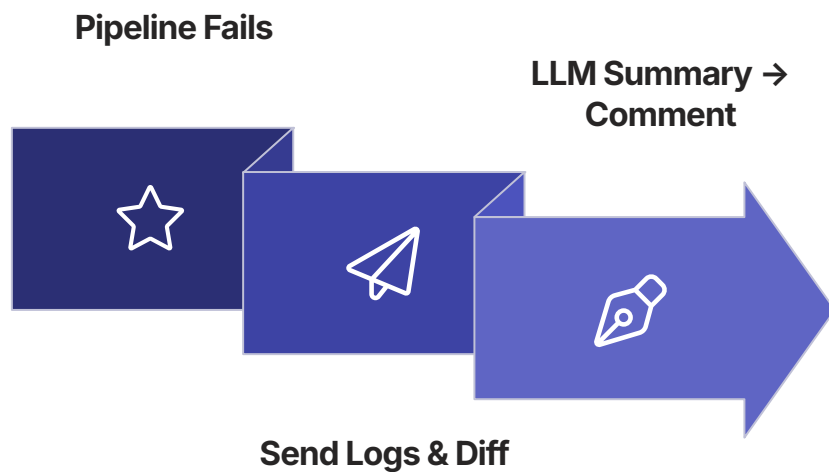
Watch — don't deploy yet

- Auto-rollback on verified regression
- Self-healing pipelines with fix-forward commits
- Agent-driven dependency upgrades

Rule of thumb: Run column 1 everywhere. Pilot column 2. Watch column 3.

Intelligent CI/CD — In Practice

CI: AI-Summarized RCA on Failures



Advisory only, ~50 lines of glue code.

CD: AI Review of terraform plan Before Apply

- terraform plan → JSON → LLM risk assessment on the PR
- Catches fuzzy stuff: scope creep, suspicious patterns, "PR description doesn't match actual change"
- Pair with policy-as-code (OPA/Rego, Kyverno) for hard rules
- Human still clicks apply



The pattern: LLM for judgment · policy-as-code for rules · human for the verb.

Demo 1 — Persona-Aware Troubleshooting

Scenario: Production K8s cluster showing high pod restart rates on a critical service.

Same incident, four lenses:

Junior View

"Explain step-by-step, include common causes and initial commands."

Senior View

"Analyze recent deploys and correlate with similar incidents. Skip the basics."

Security View

"Scan for unusual network activity, failed auth, privilege escalations."

Business View

"Customer impact? ETA to resolution?"



Takeaway: Same LLM, same data, wildly different value depending on the question. **Prompt engineering is a DevOps skill now.**



Demo 2 — k8sgpt + kubectl-ai Loop

Scenario

A deployment broke. Diagnose and fix without `kubectl describe`-ing through walls of YAML.

The Flow

What to Notice

Two binaries, no framework

Every step is a visible, deterministic command

Human stays in the loop for the actual change

Reproducible on your laptop tonight



The graduation path: Everything you just saw, running 24/7 inside your cluster as a declarative K8s resource — that's Kagent. Worth a look once this loop becomes routine.

01

`kubectl get pods` → see `ImagePullBackOff`

02

`k8sgpt analyze --explain` → plain-English diagnosis

03

`kubectl-ai "fix the nginx deployment, image tag is wrong"`

04

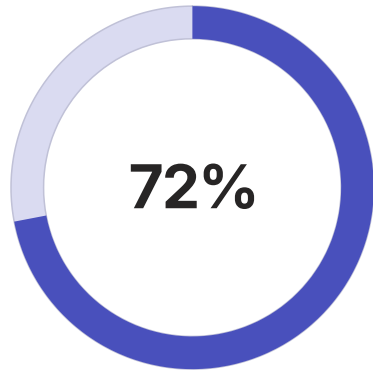
Review proposed command, approve `[y/N]`

05

`kubectl get pods` → Running

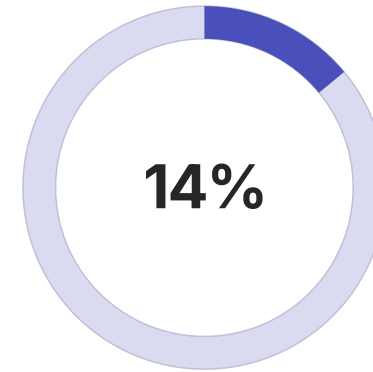
Reality + Governance

The Hype vs. The Data



Testing AI Agents

Large enterprises using or testing AI agents (*Zapier survey, Oct 2025, n=525*)



Production Scale

Have scaled an agent to production-grade operation (*survey of 650 tech leaders, Mar 2026*)

Why the Gap?

- Auditability and governance aren't mature
- "Who approves a `kubectl delete`?" is an unsolved org question
- Context engineering is the new bottleneck

Practical Guardrails

 **Human-in-the-Loop**
Required for anything destructive

 **Audit Trails**
On every tool call, no exceptions

 **Least-Privilege Service Accounts**
One per agent, scoped tightly

 **Start Read-Only**
Graduate with approval gates



Getting Started — Implementation Roadmap

1

Week 1: Assessment & Quick Wins

- Audit your top 3 time-consuming tasks
- Install k8sgpt + kubectI-ai on a dev cluster
- Try CI RCA summaries on one pipeline

Goal: Save 30 min/person in the first week

2

Month 1: Pilot

- Pick one: AI-reviewed terraform plan OR k8sgpt + kubectI-ai in staging
- Build one Slack/PR-bot integration

Goal: 20% faster incident response *or* 50% more runbook coverage

3

Quarter 1: Scale

- Expand successful pilots
- Build internal RAG corpus (runbooks + past incidents)
- Evaluate Kagent for multi-user, in-cluster operation
- Establish agent governance policy before write access goes live

Goal: 80% team adoption of AI-assisted workflows

Key Takeaways

1

AI Amplifies, Not Replaces

AI amplifies human capabilities — it doesn't replace judgment.

2

MCP Changed the Game

It's the protocol that makes agents portable, auditable, and composable.

3

Start Simple, Graduate Smart

Start with k8sgpt + kubectl-ai.
Graduate to Kagent when you outgrow shell scripts.

4

The Safety Spectrum

Run advisory AI everywhere. Pilot semi-autonomous.
Watch autonomous.

5

Governance First

Ethics and governance aren't afterthoughts — they're prerequisites to production.

AI Knowledge + Human Experience = Infinite Possibility

Connect to continue the conversation:

 **Website**

KEDARKULKARNI.in

 **Email**

KCD2026@KEDARKULKARNI.in

 **LinkedIn**

linkedin.com/in/kkulkar3

Slides, demo repos, and references will be posted after the talk.

Thank you, KCD Texas! 🧑

